🚀 Lecture 02.11: Type Conversion & Casting Mechanics

📌 Executive Architectural Summary

Type conversion translates a value from one data type representation to another. In Python, this bifurcates into **Implicit Type Conversion (Coercion)** and **Explicit Type Casting**.

🔬 Core Engineering Mechanics

1. Implicit Conversion (Widening / Promotion)

CPython automatically promotes operands to the wider data type to prevent precision loss.

> $$extint + extfloat \implies extfloat \quad (exte.g.,5 + 3.0 = 8.0)$$

2. Explicit Conversion (Type Constructors)

The programmer explicitly calls the type constructor function:

- `int(x)` : Truncates towards zero for floats; parses valid integer numeric strings.
- `float(x)` : Converts integer or numeric string to IEEE-754 64-bit float.
- `str(x)` : Invokes the object's `__str__()` method.
- `bool(x)` : Returns `False` for zero, empty collections, or `None` ; `True` otherwise.

💻 Production Python Implementation

```
# 1. Implicit Promotion
a = 5     # int
b = 2.5   # float
c = a + b # automatically promoted to float
print(f"{a} + {b} = {c} (Type: {type(c).__name__})")

# 2. Explicit Casting
num_str = "45"
num_int = int(num_str)
num_float = float(num_str)
```

```
num_back_to_str = str(num_float)

print(f"Original: {num_str} ({type(num_str).__name__})")
print(f"Int cast: {num_int} ({type(num_int).__name__})")
print(f"Float cast: {num_float} ({type(num_float).__name__})")
print(f"String cast: {num_back_to_str} ({type(num_back_to_str).__name__})")

# Truncation behavior in int()
pi_approx = 3.999
print(f"int({pi_approx}) = {int(pi_approx)} (Truncated towards zero)")
```